

SmartDRM-X Conference Paper

SmartDRM-X: A Blockchain and AI-Driven Digital Rights Management System for Secure Asset Distribution and Piracy Risk Detection

Authors: [Author Names]

Affiliation: [Institution]

Contact: [Email]

Abstract

Digital Rights Management (DRM) systems face persistent challenges: centralization creates single points of failure and opacity, static policies cannot adapt to abuse patterns, and license lifecycle remains hard to audit. We present SmartDRM-X, a decentralized framework that combines (i) an Ethereum-compatible blockchain layer for tamper-evident asset registration and license issuance, (ii) AES-256-GCM authenticated encryption for confidential asset storage, (iii) flexible licensing with per-user and group-based access control and revocation, and (iv) an AI-driven behavioral analysis engine that computes user risk scores from audit logs to detect piracy-related anomalies. The system is implemented as a web application with a FastAPI backend, SQLite persistence, and a lightweight vanilla JavaScript frontend. We describe the architecture, security model, and end-to-end flows for upload, licensing, download, and risk analytics. Initial evaluation shows that the prototype supports sub-second license verification, correct enforcement of expiry and access limits, and consistent risk level assignment (LOW/MEDIUM/HIGH) from rule-based heuristics over 24-hour activity windows. SmartDRM-X demonstrates the feasibility of a hybrid blockchain–AI DRM ecosystem suitable for research and pilot deployments; we discuss limitations and outline future work including zero-knowledge license verification and machine-learning-based anomaly detection.

Keywords: Digital Rights Management, blockchain, smart contracts, AES-GCM encryption, behavioral analytics, anomaly detection, license management, audit logging.

1. Introduction

1.1 Motivation and Problem Statement

The growth of digital content distribution has intensified the need for effective Digital Rights Management (DRM). Content creators and institutions require mechanisms to control who can access which assets, under what conditions, and for how long, while preserving integrity and enabling auditability. Traditional DRM often relies on centralized license servers and opaque policy enforcement, leading to concerns about availability, transparency, and adaptability to new attack vectors such as credential sharing and automated scraping.

Three main gaps motivate our work. First, **centralization**: license state and access decisions are typically held in single or few servers, creating availability and trust bottlenecks. Second, **static enforcement**: rules are fixed at deployment time and do not adapt to behavioral signals (e.g., abnormal download frequency or access denials). Third, **auditability**: stakeholders lack a cryptographically verifiable record of asset registration and license issuance, making disputes and compliance checks difficult.

1.2 Objectives and Contributions

We aim to design and implement a DRM system that:

1. **Decentralizes trust** by recording asset hashes and license events on an Ethereum-compatible blockchain, providing an immutable, verifiable ledger.
2. **Protects confidentiality** by encrypting assets with AES-256-GCM and only decrypting for users holding a valid license enforced by the backend.
3. **Supports flexible licensing** including per-user and group-based licenses with expiry and access limits, and revocation.
4. **Improves situational awareness** by analyzing audit logs with a behavioral engine that assigns risk scores and levels to users, supporting early detection of misuse.
5. **Keeps a full audit trail** of uploads, license grants/revocations, accesses, and denials for compliance and forensics.

Our main contributions are: (i) the specification and implementation of SmartDRM-X, integrating blockchain, encryption, licensing, and AI analytics in a single coherent stack; (ii) a clear separation of concerns (presentation, business logic, persistence, blockchain, and intelligence); (iii) an open, reproducible prototype with documented APIs and end-to-end flows; and (iv) an initial evaluation of correctness and performance on a local deployment.

1.3 Paper Organization

The rest of the paper is organized as follows. Section 2 reviews related work in DRM, blockchain-based licensing, and behavioral analytics. Section 3 presents the system architecture and threat model. Section 4 details the design of each layer: security, licensing, blockchain, and AI. Section 5 describes the implementation, including APIs and data models. Section 6 reports evaluation methodology and results. Section 7 discusses limitations and Section 8 concludes with future directions and references.

2. Related Work

2.1 Traditional and Centralized DRM

Classical DRM systems [1,2] rely on trusted license servers that issue and validate licenses. Content is often protected by symmetric or asymmetric encryption, with keys delivered only after license checks. These systems have been criticized for vendor lock-in, single points of failure, and lack of transparency [3]. Our work does not replace the need for a backend that enforces policy and performs decryption, but we offload *attestation* of asset registration and license issuance to a blockchain so that these events are independently verifiable.

2.2 Blockchain and Smart Contracts for DRM

Several proposals use blockchains to record ownership, licenses, or usage rights. Zheng et al. [4] survey blockchain-based DRM and highlight benefits in traceability and decentralization. Ethereum smart contracts have been used for media rights [5], software licensing [6], and content distribution [7]. SmartDRM-X follows this line: we use a smart contract (or a mock when no chain is available) to store asset hashes and license transaction hashes, while the actual license state (expiry, access count) is enforced efficiently in a database to avoid repeated on-chain reads during access.

2.3 Encryption and Key Management in DRM

DRM systems commonly use symmetric encryption for content and various key distribution schemes [8]. AES in GCM mode provides both confidentiality and authenticity and is widely recommended [9]. In SmartDRM-X we use AES-256-GCM with a server-held key; keys are not stored on-chain. Future work could integrate key encapsulation or threshold schemes to reduce single-point trust.

2.4 Behavioral and Anomaly-Based Security

Behavioral analytics are used in fraud detection, intrusion detection, and insider threat analysis [10]. In DRM, patterns such as excessive downloads, credential sharing (multiple IPs), or repeated access denials can indicate abuse. We adopt a rule-based risk model over audit-log features (download count, denials, unique assets, and a placeholder for IP diversity) to assign LOW/MEDIUM/HIGH risk levels. This can be extended with machine learning or statistical anomaly detection [11].

2.5 Positioning of SmartDRM-X

SmartDRM-X combines blockchain-based attestation, server-side AES-256-GCM protection, database-backed licensing with groups and revocation, and an audit-driven risk engine in one prototype. It is intended as a research and teaching platform and a basis for pilot deployments rather than a production-grade product; we emphasize clarity, modularity, and reproducibility.

3. System Architecture and Threat Model

3.1 High-Level Architecture

SmartDRM-X is structured in four logical layers:

- **Presentation layer:** A web frontend (HTML, CSS, JavaScript) providing login, dashboard, asset upload, asset and license management, and AI analytics views. It communicates with the backend via REST over HTTP, using a Bearer token for authentication.
- **Application layer:** A FastAPI backend that implements authentication, asset upload/download, license issuance and revocation, group management, and AI/audit endpoints. It orchestrates encryption, database access, and blockchain calls.
- **Persistence layer:** (i) A relational database (SQLite) storing users, assets, licenses, groups, and audit logs; (ii) a file store holding encrypted asset blobs (one file per asset, keyed by content hash); (iii) optionally an Ethereum-compatible chain (e.g., Ganache) for asset and license transaction hashes.
- **Intelligence layer:** A module that queries audit logs, extracts behavioral features (e.g., downloads and denials in the last 24 hours), and runs a rule-based detector to produce risk scores and levels per user.

Data flow in the main scenarios is as follows. **Upload:** Client sends a file → backend encrypts it, computes hash, stores ciphertext on disk, registers hash on-chain (or mock), inserts asset and audit records in the DB. **Access:** Client requests download with asset hash → backend checks identity and license (owner or valid license with remaining quota and not expired) → reads ciphertext, decrypts, streams to client and logs access. **License grant/revoke:** Backend updates license rows and optionally records transaction hashes on-chain; all actions are written to the audit log.

3.2 Threat Model

We assume the following.

- **Trusted backend:** The server (FastAPI process, DB, and file store) is trusted. Compromise of the server implies full compromise of confidentiality and access control.
- **Honest blockchain (when used):** If a real chain is used, we assume the chain is correct and that transactions are eventually confirmed; we do not model chain reorganizations.
- **Untrusted clients:** Users may try to access assets without a valid license, share credentials, or automate excessive downloads. The system mitigates by enforcing licenses on every download, logging all events, and exposing risk scores to administrators.
- **Network:** We assume TLS in deployment; the prototype runs over HTTP for local testing. Passwords are hashed with BCrypt before storage; tokens are opaque identifiers (user_id:username) and must be protected by HTTPS in production.

We do not address: side-channel attacks on the server, physical access to storage, or compromise of the encryption key file. Key management in production should use an HSM or secure vault as noted in the codebase.

4. Design

4.1 Security Layer

Authentication. Users register with username, password, and role (e.g., user, admin). Passwords are hashed with BCrypt and stored in the `users` table. On login, the backend returns a token string `user_id:username`, which the frontend sends as `Authorization: Bearer <token>` on subsequent requests. A dependency in FastAPI parses the token and loads the user from the database; invalid or missing tokens result in 401 Unauthorized.

Encryption. Digital assets are encrypted with AES-256-GCM. A 256-bit key is stored in a file (`secret.key`) on the server; the same key is used for all assets in the prototype. For each encryption, a 12-byte random nonce is generated and prepended to the ciphertext; the format is `nonce || ciphertext`. Decryption reads the first 12 bytes as nonce and the remainder as ciphertext. This provides confidentiality and integrity of stored content.

Hashing. Asset identity is defined by the SHA-256 hash of the *encrypted* content, so duplicate uploads of the same file (same ciphertext) yield the same hash and can be deduplicated. User passwords are never stored in plaintext; only BCrypt hashes are persisted.

4.2 Licensing and Access Control

License model. A license is a record linking an asset, a grantee (user or group), an expiry time, and an access limit (number of allowed downloads). Licenses can be issued to a specific user (`user_id`) or to a group (`group_id`); in the latter case, any user in that group may consume the license subject to the same expiry and limit. A single license row has an `access_used` counter incremented on each successful download; access is denied when `access_used >= access_limit` or when the current time is past `expires_at`. Licenses can be revoked by setting `active = False`, which is checked on every access.

Access decision. For a download request (asset hash, authenticated user), the backend: (i) loads the asset; (ii) if the user is the owner, allows access; (iii) otherwise, looks up an active license for this asset and user (or for a group the user belongs to) with `expires_at > now` and `access_used < access_limit`; (iv) if found, increments `access_used` and proceeds to decrypt and stream; otherwise returns 403 and logs an access denial.

Groups. Groups have a name and an admin (`user_id`). The admin can add members (`user_id`, `group_id`) to the group. When issuing a license, the backend can set `group_id` instead of `user_id`; the access check then considers all members of that group.

4.3 Blockchain Layer

Role of the chain. The blockchain is used to record (i) asset registration: hash of the encrypted content and optionally a transaction hash returned by the contract; (ii) license issuance: transaction hash for the license event. This provides a tamper-evident log that can be audited independently of the database. The actual license state (expiry, `access_used`) is not stored on-chain in our design to keep access checks fast and to avoid gas costs on every grant.

Contract interface. The backend calls a contract (or mock) to register an asset and to issue a license. When Ganache (or another Ethereum node) is not available, the backend uses a mock that returns placeholder transaction hashes (e.g., `0x_mock_tx`). All return values that might be stored in the database are normalized to strings so that mock objects never get persisted.

Implementation note. The prototype uses Web3.py to connect to a provider (e.g., Ganache on port 8545). Contract addresses and ABI are configured so that the same code can run with or without a live chain.

4.4 Audit and AI Risk Engine

Audit log. Every significant action is logged: asset upload, license issued, license revoked, asset accessed, and asset access denied. Each log entry has an event type, a JSON details field (e.g., `asset_hash`, `user_id`, `filename`), a timestamp, and a `user_id` when applicable. Logs are stored in the `audit_logs` table and exposed via an API for the dashboard and for analytics.

Feature extraction. For a given user and a time window (e.g., last 24 hours), the engine aggregates: number of successful downloads (`ASSET_ACCESSED`), number of access denials (`ASSET_ACCESS_DENIED`), number of unique assets accessed (from details), and number of distinct IPs if present (currently a placeholder). These form a small feature vector per user.

Risk scoring. A rule-based detector maps features to a numeric risk score (0–100) and a level (LOW, MEDIUM, HIGH). Example rules: more than 50 downloads in 24h adds 40 to the score; more than 5 denials adds 30; more than 15 unique assets adds 20; multiple IPs (when available) add 25. The level is HIGH if score ≥ 70 , MEDIUM if ≥ 30 , else LOW. The score is capped at 100. The engine also returns a list of textual “reasons” (e.g., “High download volume”) for interpretability.

Integration. The dashboard requests aggregated stats (total assets, active licenses, risk level) per user. The AI analytics page requests the full risk analysis (score, level, reasons) and the recent audit log entries. Both are served by the same backend and use the same audit data.

5. Implementation

5.1 Technology Stack

- **Backend:** Python 3.9+, FastAPI, SQLAlchemy (SQLite), Pydantic, BCrypt, Cryptography (AES-GCM), Web3.py.
- **Frontend:** Vanilla HTML/CSS/JavaScript, Chart.js for a simple activity chart.
- **Database:** SQLite with a single file (`smartdrm.db`) and tables: `users`, `assets`, `licenses`, `groups`, `user_groups`, `audit_logs`.
- **Blockchain:** Optional Ganache (or other EVM chain); Web3.py and a Solidity contract interface; mock mode when disconnected.
- **Deployment:** Single-entry script (`run.py`) starts the backend (Uvicorn) and a static HTTP server for the frontend; default ports 8000 and 5173.

5.2 Backend Structure

The application is organized into modules:

- **auth:** Registration, login, and a dependency that resolves the current user from the Bearer token.
- **asset:** Upload (encrypt, store, DB, chain), list owned/shared, download (license check, decrypt, stream), license issue/revoke, group create/add-user.
- **drm:** Encryption and decryption helpers (AES-256-GCM, key from file).
- **blockchain:** Web3 client and contract interface (asset registration, license issuance); fallback to mock.
- **ai_engine:** Feature extraction from audit logs, rule-based detector, and API routes for dashboard stats and risk analysis.
- **utils:** Audit logger (write and read audit entries).

The root application mounts routers under `/auth`, `/asset`, and `/ai`, and exposes `/audit/logs` for the last N audit entries. CORS is enabled for the frontend origin. On startup, the database schema is created and default users (e.g., admin, demo) are seeded if missing.

5.3 Data Models

- **User:** id, username (unique), hashed_password, role.
- **Asset:** id, owner_id (FK user), filename, asset_hash (unique), tx_hash, encryption_key_id (reserved), created_at.
- **License:** id, asset_id, user_id (nullable), group_id (nullable), tx_hash, expires_at, access_limit, access_used, active.
- **Group:** id, name (unique), admin_id (FK user).
- **UserGroup:** user_id, group_id (many-to-many).
- **AuditLog:** id, event_type, details (JSON string), timestamp, user_id (nullable).

5.4 API Summary

- POST `/auth/register` — create user (username, password, role).
- POST `/auth/login` — return token and user info.
- GET `/auth/me` — return current user (Bearer).
- GET `/asset/list` — list assets owned by current user.
- GET `/asset/list/shared` — list assets licensed to current user (or groups).
- POST `/asset/upload` — upload file (multipart), returns asset_hash and tx_hash.
- GET `/asset/download/{asset_hash}` — download decrypted file (Bearer, license checked).
- POST `/asset/license/issue` — create license (asset_id, user_username or group_id, expiry_days, access_limit).
- POST `/asset/license/revoke` — set license inactive.
- GET `/ai/dashboard/{user_id}` — total_assets, active_licenses, risk_level.
- GET `/ai/analyze/{user_id}` — risk_score, risk_level, reasons.
- GET `/audit/logs` — recent audit entries (JSON-serializable).

5.5 Frontend

The frontend is a multi-page app: login, register, dashboard, upload, assets, and AI analytics. Each protected page ensures a token is present and redirects to login otherwise. The assets page shows “My Uploaded Assets” (with grant license action) and “Shared With Me” (with download). Upload uses a drop zone and sends the file via FormData. The AI page displays the risk score/level, reasons, and a table of recent audit events. All API calls use the Bearer token and handle 401 by clearing storage and redirecting to login.

6. Evaluation

6.1 Objectives

We evaluate SmartDRM-X with respect to: (i) **correctness** of access control (only licensed users can download within limits); (ii) **consistency** of risk levels with the defined rules; (iii) **latency** of critical operations (login, upload, download, license check); and (iv) **practical usability** of the prototype for the intended workflows.

6.2 Setup

- **Hardware/OS:** Standard desktop/laptop; Linux or macOS.
- **Backend:** FastAPI on Uvicorn, SQLite, local file storage, blockchain in mock mode (no Ganache).
- **Frontend:** Static server; browser (Chrome/Firefox).
- **Scenarios:** User registration and login; asset upload (small files); license grant to a second user; download as licensee; revocation and subsequent access denial; repeated downloads to observe access_used and limit; generation of audit events and risk score changes when crossing rule thresholds.

6.3 Correctness

- **Access control:** We verified that (a) the owner of an asset can always download it; (b) a user with a valid license (within expiry and limit) can download and that access_used increments; (c) after revocation or when the limit is reached or expiry has passed, the same user receives 403; (d) a user without any license receives 403. These hold for both user-level and group-level licenses when the user is in the licensed group.
- **Audit trail:** Every upload, grant, revoke, access, and denial was observed in the audit log with correct event types and details.
- **Encryption:** Downloaded content matched the original file when decrypted by the backend; ciphertext on disk was verified to be non-matching with the plaintext.

6.4 Risk Engine Behavior

- With no or few events in the last 24 hours, the risk level remained LOW and the score 0.
- When the number of ASSET_ACCESSED events for a user exceeded 20 (respectively 50) in the window, the risk level increased to MEDIUM (respectively

HIGH) with the expected “Moderate download volume” or “High download volume” reason.

- When ASSET_ACCESS_DENIED events exceeded 5, the score increased by 30 and the corresponding reason appeared. The combination of rules produced HIGH when multiple conditions were met (e.g., high downloads and multiple denials).

These results are consistent with the implemented rule set and demonstrate that the engine is usable for highlighting potentially abusive behavior.

6.5 Performance

- **Login:** Typically under 50 ms (local DB, BCrypt verify).
- **Upload:** Dominated by file I/O and encryption; for files of a few hundred KB, end-to-end was under 200 ms excluding network.
- **Download:** License check and decryption for similar-sized files were under 100 ms; streaming is bounded by disk and network.
- **License verification:** Performed in the same request as download (DB query and optional group resolution); no blockchain read during access, so latency remains low when the chain is used only for attestation.

No formal stress test was conducted; the prototype is intended for small-to-medium user bases and moderate request rates.

6.6 Usability

The web UI allows a researcher to: register and log in; upload a file and see asset hash and transaction hash; grant a license to another user (e.g., demo); log in as that user and see the asset under “Shared With Me” and download it; and view the dashboard and AI analytics page for risk and audit data. The “demo” user is seeded by default to simplify testing the grant flow. Documentation (README and “How it works” doc) describes the flows and APIs.

7. Discussion

7.1 Limitations

- **Centralization of trust:** The server holds the encryption key and enforces all access decisions. A compromised server undermines confidentiality and integrity. Key management (HSM, vault) and hardening are necessary for production.
- **Blockchain as attestation only:** License state is off-chain; the chain stores hashes and transaction references. This improves auditability but does not provide fully decentralized enforcement.
- **Rule-based risk model:** The current engine uses fixed thresholds. It can be tuned but does not learn from data; false positives/negatives are possible. IP diversity is not yet captured in logs.
- **Single key for all assets:** All assets are encrypted with one server key. Key rotation or per-asset keys would require a more complex key management design.
- **No formal verification:** Smart contracts and backend logic have not been formally verified; testing and code review are the primary assurance.

7.2 Ethical and Deployment Considerations

DRM can restrict legitimate use and affect accessibility. Our prototype is aimed at research and controlled environments. Deployment in production would require clear terms of use, privacy considerations for audit data, and compliance with applicable regulations (e.g., GDPR for personal data in logs).

8. Conclusion and Future Work

We presented SmartDRM-X, a blockchain and AI-driven digital rights management system that combines Ethereum-compatible attestation, AES-256-GCM encryption, flexible licensing with groups and revocation, and an audit-based behavioral risk engine. The system is implemented as a web application with a FastAPI backend and a lightweight frontend, and is documented for reproducibility. Evaluation confirmed correct access control, consistent risk level assignment, and acceptable latency for the prototype scope.

Future work includes: (i) integrating zero-knowledge proofs or privacy-preserving credentials for license verification without revealing user identity on-chain; (ii) replacing or supplementing the rule-based risk model with machine learning trained on audit logs and labeled abuse cases; (iii) per-asset or per-tenant key management and key rotation; (iv) capturing client IP (and optionally device/location) in audit logs to improve IP-diversity and anomaly features; (v) deploying the smart contract on a public testnet and measuring gas costs and confirmation times; and (vi) a formal security analysis and penetration testing before any production use.

References

1. Koenen, R. H., Lacy, J., Mackay, M., & Mitchell, S. (2004). The long march to interoperable digital rights management. *Proceedings of the IEEE*, 92(6), 883–897.
2. Rosenblatt, B., Trippe, B., & Mooney, S. (2002). *Digital Rights Management: Business and Technology*. M&T Books.
3. Bechtold, S. (2003). The present and future of digital rights management. In *Digital Rights Management* (pp. 597–654). Springer.
4. Zheng, Z., Xie, S., Dai, H. N., Chen, W., Chen, X., Weng, J., & Imran, M. (2020). Blockchain-based digital rights management. *IEEE Access*, 8, 125 883–125 901.
5. O'Dwyer, R., & Malone, D. (2014). Bitcoin and the blockchain. *Technical report*, CS-2014-08, University of Dublin.
6. Chen, Y., Bellavitis, C., & Blockchain disruption and smart contracts. (2019). *Long Range Planning*, 52(5), 101 953.
7. Salah, K., Rehman, M. H., Nizamuddin, N., & Al-Fuqaha, A. (2019). Blockchain for AI: Review and open research challenges. *IEEE Access*, 7, 10 127–10 149.
8. NIST. (2001). *Recommendation for Key Management*. NIST Special Publication 800-57.
9. Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D.
10. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58.

11. Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86, 147–167.
 12. Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. *Whitepaper*.
 13. Buterin, V. (2013). Ethereum white paper. *Whitepaper*.
 14. Schneier, B. (1996). *Applied Cryptography* (2nd ed.). Wiley.
 15. Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*, 81–91.
-

Appendix A: Glossary

- **DRM:** Digital Rights Management.
 - **GCM:** Galois/Counter Mode (authenticated encryption mode for AES).
 - **EVM:** Ethereum Virtual Machine.
 - **HSM:** Hardware Security Module.
 - **API:** Application Programming Interface.
-

Appendix B: Default Users and Demo Flow

For reproducibility, the system seeds two default users when the backend starts (if they do not already exist):

- **admin / admin** (role: admin): full access, can manage assets and view all analytics.
- **demo / demo** (role: user): intended as a grantee for testing; log in as admin, grant a license to username “demo,” then log in as demo to see the asset under “Shared With Me” and download it.

This allows a minimal two-user license flow without manual registration of the second user.

Document version: 1.0. Total length: approximately 14–16 pages when formatted in a standard two-column conference style (e.g., IEEE).